

Clario

AI-Powered Real-Time Customer Support Coaching Assistant —
a multi-agent, RAG-grounded platform that coaches support
agents turn-by-turn instead of reviewing them after the fact.

FastAPI

React · TypeScript · Vite

MongoDB

ChromaDB / RAG

Groq · Llama 3.3 70B

Google Gemini

Multi-Agent Orchestration

Project Documentation — Technical Architecture & Design Report

Prepared by: **Jasir Ahamed** — Final-Year Software Engineering Intern

Repository: github.com/JasirAhamed786/Infosys-RAG-Project

Table of Contents

01	Executive Summary	3
02	Problem Statement & Objectives	4
03	System Architecture	5
04	Interaction Modes	6
05	Multi-Agent Pipeline	7
06	Technology Stack	9
07	Data Model & Database Design	10
08	RAG Knowledge Pipeline	11
09	API Reference	12
10	Frontend & Live Coaching Console	13
11	Development Timeline & Milestones	14
12	Engineering Challenges & Fixes	15
13	Evaluation Criteria Mapping	16
14	Future Enhancements & Conclusion	17

About this document

This report documents Clario as implemented in the linked repository as of the current build. It is intended for Infosys mentor review, internal design reference, and as a portfolio artifact. Diagrams are generated directly from the project's actual architecture and orchestration logic.

Executive Summary

Clario is an AI-powered real-time coaching platform built for customer support agents. Traditional support training relies on *post-call reviews* — a slow, reactive feedback loop that improves an agent's next call, never the one they are currently on. Clario inverts that model: while an agent is mid-conversation with a customer, a pipeline of six cooperating AI agents analyzes intent and sentiment, retrieves grounded knowledge-base content, drafts a response suggestion, scores escalation risk, and streams all of it into a live three-panel console — every single turn.

The system was built end-to-end across four milestones as part of an Infosys internship engagement with Vidzai Digital, and supports three distinct interaction modes so it can be evaluated even without live customers: a fully autonomous **Simulator Mode**, a human-in-the-loop **Manual Mode**, and a **Replay Mode** for coaching against historical transcripts.

6

Cooperating AI agents, each with a single, narrow responsibility, orchestrated per conversation turn.

3

Interaction modes — Simulator, Manual, and Replay — sharing one identical coaching pipeline.

2

LLM providers used deliberately: Groq for low-latency turns, Gemini for deeper reasoning tasks.

What Clario Delivers

- **In-session coaching** — tone feedback, suggested responses, and knowledge recommendations appear while the conversation is still live, not after it ends.
- **Grounded answers, not guesses** — the Knowledge Recommendation Agent retrieves from an actual vector-indexed knowledge base (ChromaDB) instead of letting the LLM hallucinate policy details.
- **Escalation foresight** — a dedicated agent continuously scores the likelihood a conversation is heading toward escalation, with reasoning attached, not just a number.
- **Closure & analytics** — every session ends with a structured performance report (sentiment journey, resolution quality score, coaching recommendations) and rolls up into a cross-session analytics dashboard.

Why this matters

Organizations lose the moment where coaching is most valuable — the live call — because human QA review only happens afterward. Clario's core bet is that a fast, narrow multi-agent pipeline can deliver useful-enough guidance *inside* that window, without needing a human reviewer in the loop at all.

Problem Statement & Objectives

The Problem

Customer service representatives handle dozens of support interactions daily, often without access to real-time guidance, relevant knowledge resources, or immediate feedback on response quality. Training agents through post-call reviews is slow and reactive, failing to improve performance during live interactions where it matters most. Organizations need intelligent systems that coach agents *in the moment* — improving resolution quality, reducing escalations, and enhancing customer experience continuously.

Project Objective

Develop an AI Customer Support Coaching Assistant that coaches agents in real time during live text-based support interactions, transforming reactive agent training into proactive in-session coaching.

Outcomes Targeted

#	OUTCOME
1	A multi-agent real-time coaching pipeline that analyzes each conversation turn and delivers instant guidance.
2	A Customer Simulator Agent generating realistic, scenario-based customer messages for self-contained testing without real users.
3	RAG-powered knowledge recommendations surfacing relevant FAQs, articles, and troubleshooting steps per turn.
4	Continuous escalation-risk detection with reasoning and recommended intervention strategies.
5	Post-interaction performance reports with sentiment journey, resolution quality scoring, and personalized improvement recommendations.

Modules Implemented

Session Configuration

Knowledge Base Ingestion + RAG

Simulator & Conversation Mgmt

Agent Orchestration Layer

Three-Panel Console UI

Escalation Alerts

Reports & Analytics

System Architecture

Clario follows a layered architecture: a React single-page frontend talks to a FastAPI backend over REST and Server-Sent Events; the backend's orchestration layer fans out to six agents backed by two LLM providers, and persists state in MongoDB (structured records) and ChromaDB (vector search over the knowledge base).

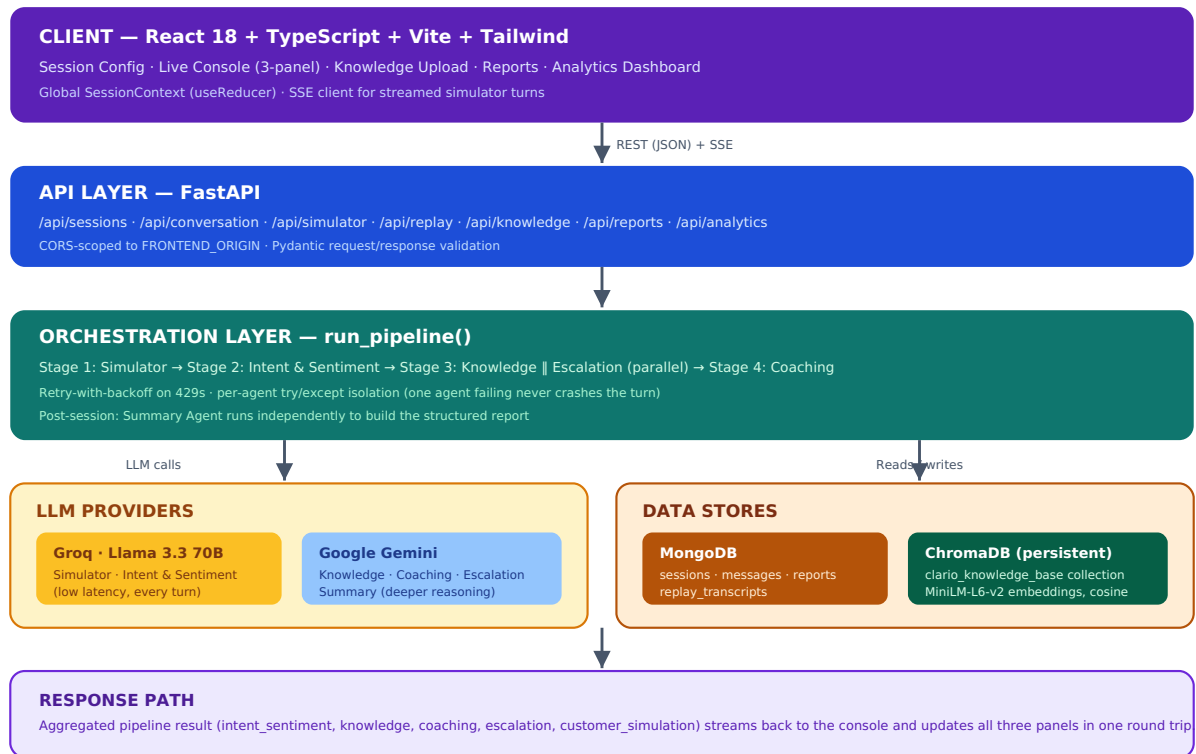


Figure 3.1 — Clario layered system architecture: client, API, orchestration, LLM providers, and data stores.

Design Principle: Two LLM Providers, Deliberately Split

Agents that must respond within the live turn budget (Simulator, Intent & Sentiment) are routed to Groq for its low inference latency. Agents that benefit more from reasoning depth than raw speed — Knowledge Recommendation, Coaching, Escalation, and the Post-Interaction Summary — are routed to Gemini. This split keeps the perceived latency of a live turn low while reserving the heavier reasoning budget for the analysis that agents actually read.

Interaction Modes

Every mode ultimately feeds the **same** orchestration pipeline — the only thing that changes is where the customer's message comes from. This was a deliberate design decision so that Simulator, Manual, and Replay sessions all produce identical, comparable coaching output and reports.

Simulator Mode

A Customer Simulator Agent generates realistic, persona-driven customer messages turn by turn from a defined scenario — lets the whole pipeline be tested end-to-end without any real customer present, with a configurable, evolving frustration level.

Manual Mode

The support agent pastes in real incoming customer text. The Simulator Agent is skipped entirely; the pasted message is treated as the authoritative customer turn and flows directly into Intent & Sentiment.

Replay Mode

A pre-existing support transcript (Customer:/ Agent: prefixed text) is uploaded and stepped through one line at a time, running the exact same coaching pipeline against historical conversations for retrospective analysis or demoing.

Why one shared pipeline matters

Because `run_pipeline()` doesn't know or care which mode supplied the customer message, coaching quality, report structure, and analytics are directly comparable across a live Manual session and a Replay session of an old transcript — there is no special-cased "replay-only" scoring logic that could silently diverge from production.

Multi-Agent Pipeline

Six agents, each with a single narrow responsibility, are orchestrated per conversation turn. A turn always resolves in four stages — the pipeline never lets one slow agent block an independent one.

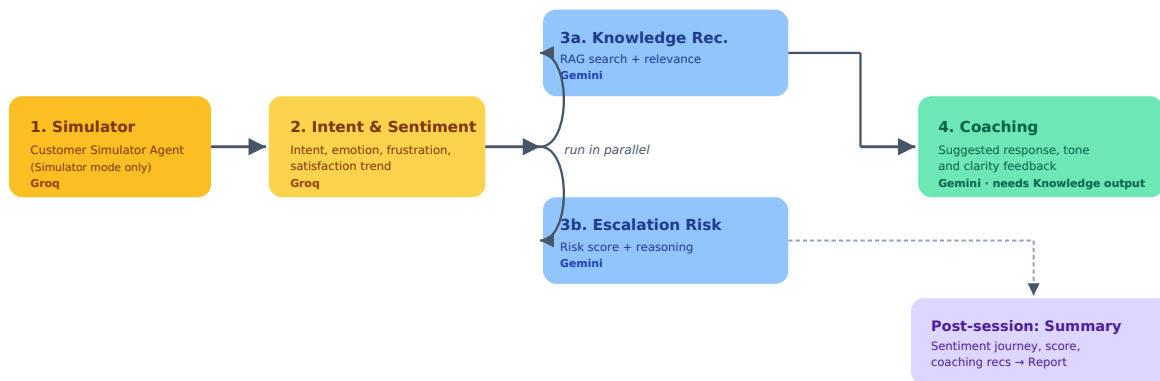


Figure 5.1 — Per-turn agent orchestration. Knowledge Recommendation and Escalation Risk run concurrently since neither depends on the other; Coaching waits on Knowledge's output so its suggested response can cite retrieved KB content.

Agent Reference

1 • Customer Simulator Agent

Groq

Generates realistic, scenario-consistent customer messages turn by turn, with a configurable persona and a frustration level that evolves across the conversation instead of resetting each turn.

2 • Intent & Sentiment Analysis Agent

Groq

Identifies customer intent (e.g. billing_issue, technical_problem, refund_request), emotional state, a numeric frustration score, and the satisfaction trend versus prior turns.

3 • Knowledge Recommendation Agent

Gemini

Queries the shared ChromaDB collection for the top-3 most relevant chunks across all uploaded knowledge documents, filters below a minimum similarity threshold, then asks Gemini for a one-sentence "why this is relevant" explanation per chunk. Runs only when the detected intent suggests an information need.

4 • Coaching & Response Suggestion Agent

Gemini

Generates a suggested agent reply grounded in the retrieved knowledge, plus tone-quality feedback and concrete communication improvement tips for the turn.

5 • Escalation Risk Monitor Agent

Gemini

Continuously scores escalation likelihood with attached reasoning and a recommended de-escalation or resolution strategy, independent of the knowledge lookup.

6 • Post-Interaction Summary Agent

Gemini

Runs once, after the session ends, over the full message history: produces an interaction summary, a turn-by-turn sentiment journey, a resolution quality score, coaching recommendations, escalation triggers, and identified knowledge gaps — persisted as the session's report.

Resilience by design

Every agent call is wrapped by `_safe_run_agent()` in the orchestration layer: if any single agent throws (rate limit, malformed JSON, network error), the pipeline logs it and substitutes a structured fallback result instead of crashing the turn — the other agents' output still reaches the console.

Technology Stack

LAYER	TECHNOLOGY	PURPOSE
Frontend framework	React 18 + TypeScript + Vite	SPA build, fast HMR dev loop, type safety across components
Styling	Tailwind CSS + a custom design-token system	Consistent spacing/color scale across the console, forms, and dashboards
Frontend state	React Context + <code>useReducer</code>	Global <code>SessionContext</code> persists session/mode/messages across page navigation
Frontend extras	Framer Motion, Lucide React, React Router	Panel/alert animations, iconography, page routing
Backend framework	FastAPI (Python)	Async-capable REST API, Pydantic request/response validation, SSE streaming
Orchestration	Custom pipeline (<code>run_pipeline()</code>) with <code>ThreadPoolExecutor</code>	Stage sequencing, controlled agent-level parallelism, retry-with-backoff
Primary database	MongoDB (PyMongo)	Sessions, messages, reports, knowledge-doc metadata, replay transcripts
Vector store	ChromaDB (persistent client)	Shared knowledge-base collection, cosine similarity search
Embeddings	sentence-transformers — <code>all-MiniLM-L6-v2</code>	Chunk + query embedding for RAG retrieval
Fast-turn LLM	Groq — Llama 3.3 70B class models	Simulator & Intent/Sentiment agents (low-latency requirement)
Reasoning LLM	Google Gemini	Knowledge, Coaching, Escalation, Summary agents (reasoning-heavy tasks)
Document parsing	pypdf	Extracting text from uploaded PDF knowledge-base documents

Data Model & Database Design

MongoDB stores five logical collections. Documents use UUID string primary keys (rather than ObjectId) so that session/message identifiers created on the frontend before a network round-trip remain stable and idempotency checks (e.g. "has this turn already been persisted?") stay simple string comparisons.

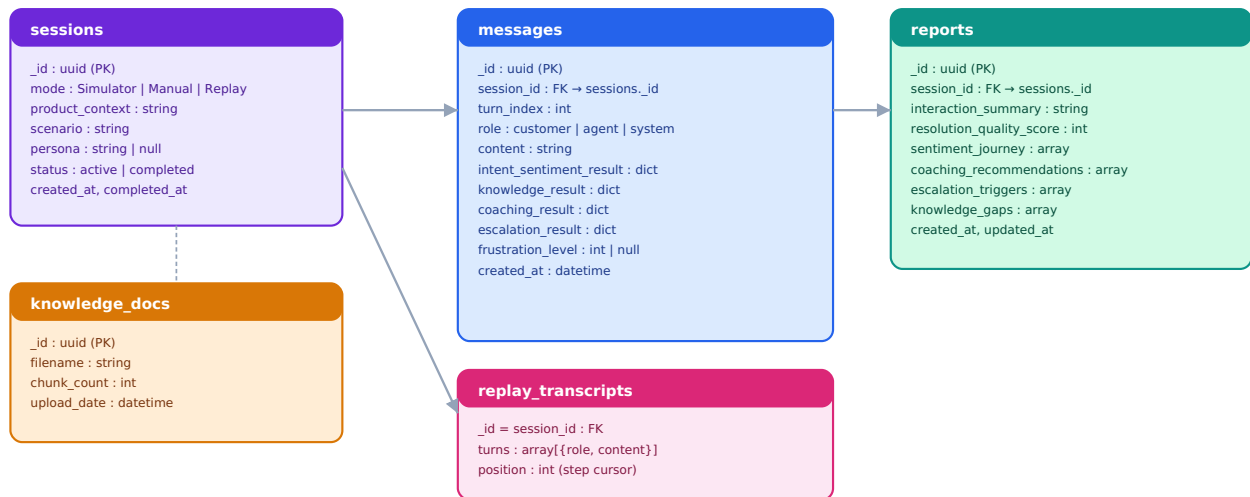


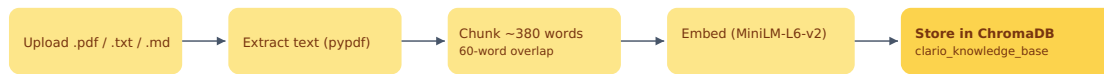
Figure 7.1 — MongoDB collections and their relationships (all foreign keys are UUID strings, not ObjectIds).

Vector Store (ChromaDB)

All uploaded knowledge documents are chunked (~380 words, 60-word overlap) and embedded into a single shared collection, `clario_knowledge_base`, configured for cosine similarity. Each chunk carries `source_document`, `chunk_index`, and `upload_date` metadata, so retrieval results can always be traced back to their originating file and optionally filtered per-document.

RAG Knowledge Pipeline

INGESTION (once, per uploaded document)



RETRIEVAL (every turn that needs information)

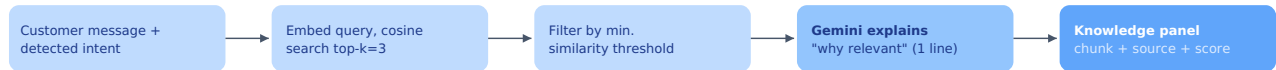


Figure 8.1 — Two-phase RAG pipeline: one-time ingestion into the shared vector collection, and per-turn retrieval + relevance explanation.

Retrieval only runs when the Intent & Sentiment Agent's detected intent falls into an information-seeking category (billing_issue, technical_problem, refund_request, how_to, account_access, etc.) — purely conversational turns skip the vector search entirely, saving latency and avoiding irrelevant "knowledge" being surfaced.

API Reference

METHOD	ENDPOINT	PURPOSE
POST	/api/sessions	Create a new coaching session (mode, product context, scenario, persona)
GET	/api/sessions	List recent sessions
GET	/api/sessions/{id}	Fetch a single session
POST	/api/conversation/turn	Run one conversation turn through the full pipeline (Manual / Simulator)
POST	/api/simulator/start	Start a Simulator-mode session's first customer message
POST	/api/simulator/message	Advance a Simulator-mode conversation by one turn
GET	/api/simulator/stream/{id}	Server-Sent Events stream of a simulator-generated reply, token by token
POST	/api/replay/upload	Upload a transcript file for Replay mode
GET	/api/replay/status/{id}	Current step position of an uploaded replay transcript
POST	/api/replay/next	Advance the replay transcript by one line, running the pipeline on customer turns
POST	/api/knowledge/upload	Upload and ingest a knowledge-base document (chunk + embed + store)
POST	/api/knowledge/query	Ad-hoc retrieval query against the knowledge base
GET	/api/knowledge/stats	Chunk count / collection stats
DELETE	/api/knowledge	Clear the shared knowledge collection
GET	/api/coaching/feed	Coaching feed for a session at a given turn index
GET	/api/escalation/alerts	Current escalation alerts
POST	/api/reports/generate/{session_id}	Run the Summary Agent and persist the post-interaction report
GET	/api/reports/{session_id}	Fetch a session's report
GET	/api/reports	List all completed reports
GET	/api/analytics/dashboard	Cross-session analytics rollout (avg. resolution score, top escalation triggers, knowledge gaps)
GET	/api/health	Liveness check

Frontend & Live Coaching Console

The frontend is a single-page app with a persistent left sidebar and a top nav, built around one central screen — the Live Coaching Console — flanked by supporting pages for setup, knowledge management, and post-session review.

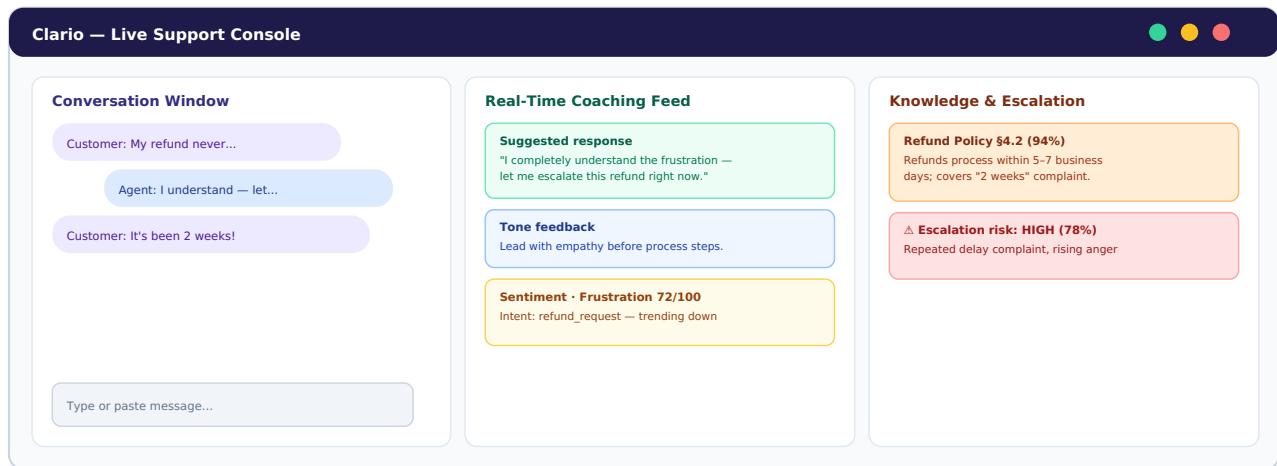


Figure 10.1 — Three-panel Live Coaching Console: conversation, real-time coaching feed, and knowledge/escalation panel, all updating after every exchange.

Page Map

PAGE	RESPONSIBILITY
Home	Entry point and mode selection
Session Configuration	Pick interaction mode, define product context, scenario, and persona
Live Console	The three-panel coaching interface described above; handles Simulator, Manual, and Replay turns
Knowledge Base Upload	Upload and manage documents ingested into the shared ChromaDB collection
Reports	Generate and browse post-interaction reports with sentiment journey and coaching recommendations
Analytics Dashboard	Cross-session trends — average resolution score, top escalation triggers, recurring knowledge gaps

State Management

A global `SessionContext` (React `useReducer`) holds the active session, its messages, and pipeline results, so navigating between pages (e.g. from the Console to the Knowledge Base and back) doesn't lose conversation state. Optimistic UI updates render the agent's own message immediately, before the pipeline response returns, so the console never feels like it's waiting on the network.

Development Timeline & Milestones



Figure 11.1 — Four-milestone build plan (8 weeks) plus an ongoing hardening pass.

Engineering Challenges & Fixes

Beyond feature delivery, a significant part of this internship was diagnosing and fixing subtle correctness bugs that only surface under real multi-turn usage. The most instructive ones are documented below — these are the kind of issues that come up in a code walkthrough or viva.

Bug — Turn-ordering race condition (sentiment lag)

The Simulator and Intent & Sentiment agents originally ran concurrently. In Simulator mode this meant Intent & Sentiment analyzed the *previous* turn's stored customer message while the Simulator generated a brand new one in parallel — so the sentiment panel shown next to a customer's message bubble was always one turn behind.

Fix — Sequenced Stage 1: the Simulator now always runs first and its fresh output is fed directly into Intent & Sentiment before that stage starts, guaranteeing they analyze the same message.

Bug — Turn-index collisions in generated reports

Report generation and the underlying message store both keyed off `turn_index`, but under certain replay/retry paths the same index could be written twice, silently overwriting one turn's analysis with another's.

Fix — Idempotent inserts: every write path now checks for an existing message at that `(session_id, turn_index, role)` tuple before inserting, and updates in place instead of duplicating.

Bug — Markdown bleed-through in coaching agent responses

The Coaching Agent's raw LLM output occasionally included Markdown formatting (bold asterisks, bullet dashes) that rendered as literal characters in the console instead of being interpreted, cluttering the suggested-response text.

Fix — Added prompt constraints requiring plain text output, plus a light server-side sanitation pass stripping stray Markdown tokens before the response reaches the frontend.

Design review finding — LLM provider / documentation drift

A later architecture review found the Coaching and Escalation agents had drifted onto the Groq client during debugging, while the documented design (and the original rationale — Gemini capacity reserved for reasoning-heavy agents) called for Gemini. Caught before final submission via a full-repo review comparing documented architecture against actual agent imports.

Fix — Routed Coaching and Escalation back onto dedicated Gemini model settings, and corrected the model identifier used by the Knowledge Recommendation Agent to a currently-valid Gemini model.

Hygiene finding — Vector store committed to version control

The ChromaDB persistence directory name in `.gitignore` didn't match the actual configured path, so ~2 MB of binary vector-index files were being tracked in git history.

Fix — Corrected the ignore pattern and untracked the generated index; it now regenerates locally from the ingested documents on first run.

Evaluation Criteria Mapping

Mapped against the criteria defined in the original project brief:

EVALUATION CRITERION	HOW CLARIO ADDRESSES IT
Realism & scenario consistency of Simulator Agent output	Persona + scenario + evolving frustration level are carried through every generation call, with the last 6 turns of context included in each prompt.
Accuracy of Intent & Sentiment across emotional states	Structured JSON output (intent, emotion, frustration_score, satisfaction_trend) constrained by prompt schema, validated per-turn.
Relevance & retrieval quality of Knowledge Agent	Cosine-similarity vector search with an explicit minimum similarity threshold, plus an LLM-generated relevance explanation per result.
Usefulness of coaching, suggestions, and escalation alerts	Coaching suggestions are grounded in retrieved knowledge (not free-floating), and escalation scoring always carries reasoning, not just a number.
Quality & clarity of post-interaction reports	Structured report schema — summary, turn-by-turn sentiment journey, resolution score, and categorized recommendations — plus a cross-session analytics dashboard.
Completeness of implementation, testing, docs, demo	All three modes share one pipeline and were tested against the same scenarios; this document plus a one-page demo handout support the final presentation.

Future Enhancements & Conclusion

Potential Next Steps

- **Async data layer** — migrate the two streaming/upload endpoints from blocking PyMongo calls to a fully async driver (Motor) to remove event-loop blocking under concurrent load.
- **Per-document knowledge filtering in the UI** — surface the existing `filter_metadata` capability in the Knowledge Agent so coaches can scope retrieval to a specific manual or policy document.
- **Auth & multi-agent-seat support** — session ownership and role-based access, so a team lead can review multiple agents' sessions from one analytics view.
- **Configurable escalation thresholds** — let team leads tune what risk score triggers an alert overlay, rather than a fixed threshold.

Conclusion

Clario demonstrates that a small set of narrowly-scoped, well-orchestrated agents — rather than one large do-everything model — can deliver real-time, grounded coaching inside a live support conversation. The three interaction modes sharing a single pipeline made the system straightforward to test and demo without needing real production traffic, and the milestone structure kept each agent's scope tight enough to debug and reason about independently. The engineering-challenges section above reflects the project's real debugging history, and is intended to demonstrate not just what was built, but how correctness issues in a multi-agent, multi-provider system were found and fixed.

Project status

All four milestones are functionally complete across Simulator, Manual, and Replay modes, with post-interaction reporting and cross-session analytics in place. Remaining items are hardening and polish, tracked in the section above.